

SIMATS ENGINEERING

ASSESSMENT TOOL 1 (High)- Analytical Problem Solving

COURSE NAME: Data Structures

COURSE CODE:CSA0316

COURSE OUTCOME COVERED

CO2 : Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques. (BTL3, BTL4)

Assessment Tool Weightage: **30%**

QUESTIONS:

1. Identify if the given expressions are balanced or not. (i) $[9 * (k+8 - \{ 6 + e \} + 77) / 100,$
(ii) $((a*b) + [u/2] - d).$
2. Given a stack with repeated elements, design an approach to compress it (value & frequency) while maintaining stack operations.
 - Apply stack ADT modifications.
 - Analyze trade-off between time and space

Code of Conduct:

I G.Hima Bindu(192511377) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *G. Hima Bindu*

1. Identify if the given expressions are balanced or not. (i) $[9 * (k+8 - \{ 6 + e \} + 77) / 100]$, (ii) $((a*b) + [u/2] - d)$.

G. Himabindu
192511377

C8A 0316.

C02 - AT1

1) Identify if the given expression are balanced or not

(i) $[9 * (k+8 - \{ 6 + e \} + 77) / 100]$ G

(ii) $((a*b) + [u/2] - d)$.

Sol:

Given

$[9 * (k+8 - \{ 6 + e \} + 77) / 100]$

Step	Symbol	operation	stack
1	[	push	[
2	(	push	[(
3	{	push	[({
4	}	pop {	[(
5	)	pop (	[
6	End of Expression	[remains in stack	[

The

→ [is opened

→ (is opened and closed correctly.

→ { is opened and closed correctly.

→ The opening square Bracket has not closed.

So the, given expression

$[9 * (k+8 - \{ 6 + e \} + 77) / 100]$

is not Balanced.

(ii) $((a \times b) + [u/2] - d)$

Step	Symbol	operation	stack
		push	(
1	(		((
2	(	push	(((
3	[	push	((([
4	[	push	((([[
5	]	pop [	((([
6	)	pop (	((
7	End of Expression	Two (remain	((

→ The

- Three opening parentheses ((are present
- only one closing parenthesis) is available
- The square brackets [] are correctly matched.
- Two opening parentheses remain unmatched.

So, the Given Expression

$((a \times b) + [u/2] - d)$

is not Balanced.

2. Given a stack with repeated elements, design an approach to compress it (value & frequency) while maintaining stack operations.

- Apply stack ADT modifications.
- Analyze trade-off between time and space

Sol:

Code:

```
#include <stdio.h>
#include <stdlib.h>
```

```
#define MAX 100
```

```
// Structure for compressed stack element
```

```
typedef struct
```

```
{
    int value;
    int freq;
} StackNode;
```

```
StackNode stack[MAX];
```

```
int top = -1;
```

```
// Push operation
```

```
void push(int x)
```

```
{
    if (top == MAX - 1)
    {
        printf("Stack Overflow\n");
        return;
    }
```

```
    if (top != -1 && stack[top].value == x)
    {
        stack[top].freq++;
    }
    else
    {
        top++;
        stack[top].value = x;
        stack[top].freq = 1;
    }
}
```

// Pop operation

void pop()

```
{
    if (top == -1)
    {
        printf("Stack Underflow\n");
        return;
    }
```

```
    if (stack[top].freq > 1)
    {
        stack[top].freq--;
    }
    else
    {
        top--;
    }
}
```

// Peek operation

void peek()

```
{
    if (top == -1)
    {
        printf("Stack is Empty\n");
    }
    else
    {
        printf("Top Element = %d\n", stack[top].value);
    }
}
```

// Display compressed stack

void display()

```
{
    int i;
```

```

if (top == -1)
{
    printf("Stack is Empty\n");
    return;
}

printf("\nCompressed Stack (Top to Bottom)\n");

for (i = top; i >= 0; i--)
{
    printf("(%d, %d)\n", stack[i].value, stack[i].freq);
}
}

int main()
{
    // Push elements
    push(2);
    push(2);
    push(2);

    push(4);
    push(4);

    push(7);
    push(7);
    push(7);
    push(7);

    display();

    peek();

    printf("\nAfter one Pop:\n");
    pop();
    display();

```

```

    printf("\nAfter another Pop:\n");
    pop();
    display();

    return 0;
}

```

Output:

The screenshot shows a C program that implements a compressed stack. The code defines a structure `StackNode` with `int value;` and `int freq;`, and an array `stack[MAX]` with `top = -1`. The output on the right shows the stack state before and after a pop operation. Before the pop, the stack (top to bottom) contains (7, 3), (4, 2), and (2, 3). After the pop, the stack contains (7, 2), (4, 2), and (2, 3). The message "=== Code Execution Successful ===" is displayed at the bottom.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #define MAX 100
5
6  // Structure for compressed stack element
7  typedef struct
8  {
9      int value;
10     int freq;
11 } StackNode;
12
13 StackNode stack[MAX];
14 int top = -1;
15

```

Compressed Stack (Top to Bottom)

(7, 3)

(4, 2)

(2, 3)

After another Pop:

Compressed Stack (Top to Bottom)

(7, 2)

(4, 2)

(2, 3)

=== Code Execution Successful ===

Modified Stack ADT

- Push(x):
 - If x is the same as the top value, increase its frequency.
 - Otherwise, push a new (x,1) pair.
- Pop():
 - If top frequency > 1, decrease the frequency.
 - Otherwise, remove the top pair.
- Peek(): Return the top value.
- isEmpty(): Check whether the stack is empty.

Time and Space Trade-off:

Operation	Time Complexity	Space Complexity
Push	$O(1)$	$O(k)$
Pop	$O(1)$	$O(k)$
Peek	$O(1)$	$O(1)$

- k = number of distinct consecutive elements.

Advantages

- Reduces memory usage when many consecutive elements are repeated.
- Stack operations remain $O(1)$.
- Efficient for data with frequent duplicates.

Disadvantage

- Provides little or no space saving when there are no consecutive repeated elements.

RUBRICS FOR CALCULATION:

Numerical Problems

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation